

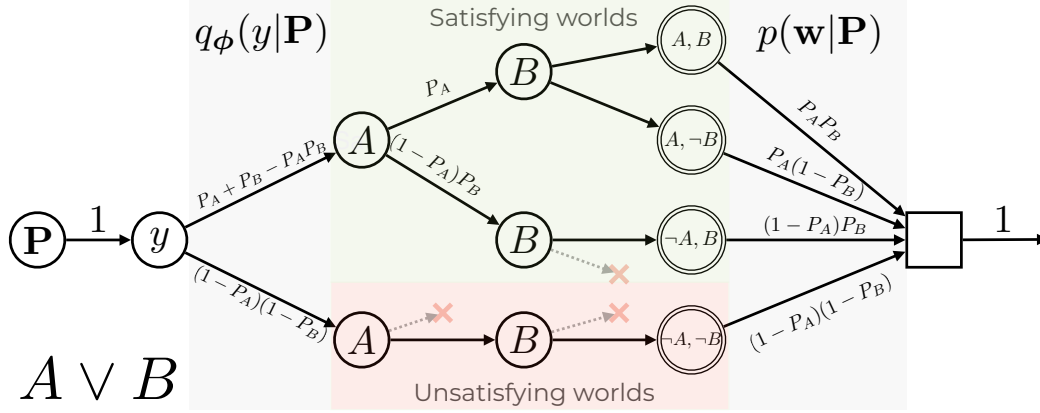


Figure 6: The tree flow network corresponding to weighted model counting on the formula $A \vee B$. Following edges upwards means setting the corresponding binary variable to true (and to false by following edges downwards). We first choose probabilities for the propositions A and B , then choose whether we want to sample a world that satisfies the formula $A \vee B$. $y = 1$ is the WMC of $A \vee B$, and equals its outgoing flow $P_A + P_B - P_A P_B$. Terminal states (with two circles) represent choices of the binary variables A and B . These are connected to a final sink node, corresponding to the prior over worlds $p(\mathbf{w}|\mathbf{P})$. The total ingoing and outgoing flow to this network is 1, as we deal with normalized probability distributions p and q_ϕ .

D A-NeSI and GFlowNets

A-NeSI is heavily inspired by the theory of GFlowNets [9, 10], and we use this theory to derive our loss function. In the current section, we discuss these connections and the potential for future research by taking inspiration from the GFlowNet literature. In this section, we will assume the reader is familiar with the notation introduced in [10] and refer to this paper for the relevant background.

D.1 Tree GFlowNet representation

The main intuition is that we can treat the inference model q_ϕ in Equation 3 as a ‘trivial’ GFlowNet. We refer to Figure 6 for an intuitive example. It shows what a flow network would look like for the formula $A \vee B$. We take the reward function $R(\mathbf{w}, \mathbf{y}) = p(\mathbf{w}, \mathbf{y})$. We represent states s by $s = (\mathbf{P}, \mathbf{y}_{1:i}, \mathbf{w}_{1:j})$, that is, the belief $\mathbf{P}$, a list of some dimensions of the output instantiated with a value and a list of some dimensions of the world assigned to some value. Actions a set some value to the next output or world variable, i.e., $A(s) = Y_{i+1}$ or $A(s) = W_{j+1}$.

Note that this corresponds to a flow network that is a tree everywhere but at the sink since the state representation conditions on the whole trajectory observed so far. We demonstrate this in Figure 6. We assume there is some fixed ordering on the different variables in the world, which we generate the value of one by one. Given this setup, Figure 6 shows that the branch going up from the node y corresponds to the regular weighted model count (WMC) introduced in Equation 1.

The GFlowNet forward distribution P_F is q_ϕ as defined in Equation 3. The backward distribution P_B is $p(\mathbf{w}, \mathbf{y}|\mathbf{P})$ at the sink node, which chooses a terminal node. Then, since we have a tree, this determines the complete trajectory from the terminal node to the source node. Thus, at all other states, the backward distribution is deterministic. Since our reward function $R(\mathbf{w}, \mathbf{y}, \mathbf{P}) = p(\mathbf{w}, \mathbf{y}|\mathbf{P})$ is normalized, we trivially know the partition function $Z(\mathbf{P}) = \sum_{\mathbf{w}} \sum_{\mathbf{y}} R(\mathbf{w}, \mathbf{y}|\mathbf{P}) = 1$.

D.2 Lattice GFlowNet representation

Our setup of the generative process assumes we are generating each variable in the world in some order. This is fine for some problems like MNISTAdd, where we can see the generative process as ‘reading left to right’. For other problems, such as Sudoku, the order in which we would like to generate the symbols is less obvious. Would we generate block by block? Row by row? Column by column? Or is the assumption that it needs to be generated in some fixed order flawed by itself?

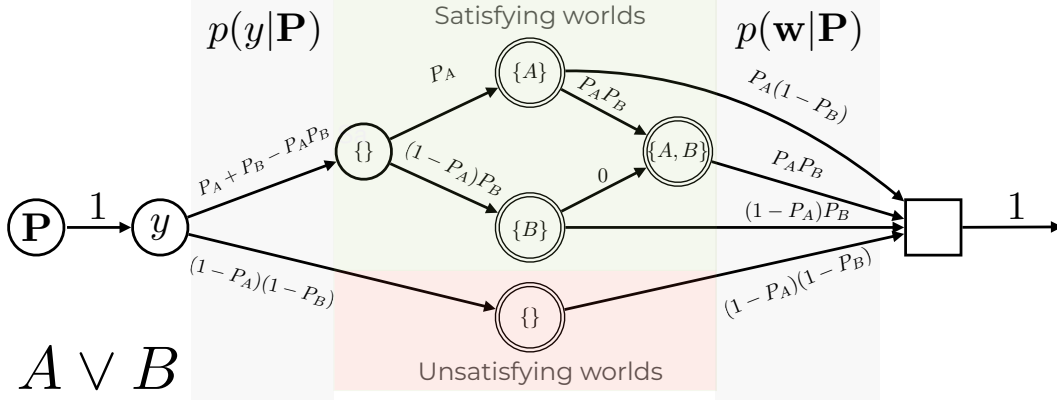


Figure 7: The lattice flow network corresponding to weighted model counting on the formula $A \vee B$. In this representation, nodes represent sets of included propositions. Terminal states represent sets of random variables such that $A \vee B$ is true given $y = 1$, or false otherwise.

In this section, we consider a second GFlowNet representation for the inference model that represents states using sets instead of lists. We again refer to Figure 7 for the resulting flow network of this representation for $A \vee B$. We represent states using $s = (\mathbf{P}, \{y_i\}_{i \in I_Y}, \{w_i\}_{i \in I_W})$, where $I_Y \subseteq \{1, \dots, k_Y\}$ and $I_W \subseteq \{1, \dots, k_W\}$ denote the set of variables for which a value is chosen. The possible actions from some state correspond to $A(s) = \bigcup_{i \notin I_W} W_i$ (and analogous for when y is not yet completely generated). For each variable in W for which we do not have the value yet, we add its possible values to the action space.

With this representation, the resulting flow network is no longer a tree but a DAG, as the order in which we generate the different variables is now different for every trajectory. What do we gain from this? When we are primarily dealing with categorical variables, the two gains are 1) we no longer need to impose an ordering on the generative process, and 2) it might be easier to implement parameter sharing in the neural network that predicts the forward distributions, as we only need a single set encoder that can be reused throughout the generative process.

However, the main gain of the set-based approach is when worlds are all (or mostly) binary random variables. We illustrate this in Figure 7. Assume $W = \{0, 1\}^{k_W}$. Then we can have the following state and action representations: $s = (\mathbf{P}, \mathbf{y}, I_W)$, where $I_W \subseteq \{1, \dots, k_W\}$ and $A(s) = \{1, \dots, k_W\} \setminus I_W$. The intuition is that I_W contains the set of all binary random variables that are set to 1 (i.e., true), and $\{1, \dots, k_W\} \setminus I_W$ is the set of variables set to 0 (i.e., false). The resulting flow network represents a *partial order* over the set of all subsets of $\{1, \dots, k_W\}$, which is a *lattice*, hence the name of this representation.

With this representation, we can significantly reduce the size and computation of the flow network required to express the WMC problem. As an example, compare Figures 6 and 7, which both represent the WMC of the formula $A \vee B$. We no longer need two nodes in the branch $y = 0$ to represent that we generate A and B to be false, as the initial empty set $\{\}$ already implies they are. This will save us two nodes. Similarly, we can immediately stop generating at $\{A\}$ and $\{B\}$, and no longer need to generate the other variable as false, which also saves a computation step.

While this is theoretically appealing, the three main downsides are 1) P_B is no longer trivial to compute; 2) we have to handle the fact that we no longer have a tree, meaning there is no longer a unique optimal P_F and P_B ; and 3) parallelization becomes much trickier. We leave exploring this direction in practice for future work.

E Analyzing the Joint Matching Loss

This section discusses the loss function we use to train the joint variant in Equation 8. We recommend interested readers first read Appendix D.1. Throughout this section, we will refer to $p := p(\mathbf{w}, \mathbf{y}|\mathbf{P})$ (Equation 5) and $q := q_{\phi, \psi}(\mathbf{w}, \mathbf{y}|\mathbf{P})$ (Equation 2). We again refer to [10] for notational background.

E.1 Trajectory Balance

We derive our loss function from the recently introduced Trajectory Balance loss for GFlowNets, which is proven to approximate the true Markovian flow when minimized. This means sampling from the GFlowNet allows sampling in proportion to reward $R(s_n) = p$. The Trajectory Balance loss is given by

$$\mathcal{L}(\tau) = \left(\log \frac{F(s_0) \prod_{t=1}^n P_F(s_t | s_{t-1})}{R(s_n) \prod_{t=1}^n P_B(s_{t-1} | s_t)} \right)^2, \quad (28)$$

where s_0 is the source state, in our case $\mathbf{P}$, and s_n is some terminal state that represents a full generation of $\mathbf{y}$ and $\mathbf{w}$. In the tree representation of GFlowNets for inference models (see Appendix D.1), this computation becomes quite simple:

1. $F(s_0) = 1$, as $R(s_n) = p$ is normalized;
2. $\prod_{t=1}^n P_F(s_t | s_{t-1}) = q$: The forward distribution corresponds to the inference model $q_\phi(\mathbf{w}, \mathbf{y} | \mathbf{P})$;
3. $R(s_n) = p$, as we define the reward to be the true joint probability distribution $p(\mathbf{w}, \mathbf{y} | \mathbf{P})$;
4. $\prod_{t=1}^n P_B(s_{t-1} | s_t) = 1$, since the backward distribution is deterministic in a tree.

Therefore, the trajectory balance loss for (tree) inference models is

$$\mathcal{L}(\mathbf{P}, \mathbf{y}, \mathbf{w}) = \left(\log \frac{q}{p} \right)^2 = (\log q - \log p)^2, \quad (29)$$

i.e., the term inside the expectation of the joint matching loss in Equation 8. This loss function is stable because we can sum the individual probabilities in log-space.

A second question might then be how we obtain ‘trajectories’ $\tau = (\mathbf{P}, \mathbf{y}, \mathbf{w})$ to minimize this loss over. The paper on trajectory balance [33] picks τ *on-policy*, that is, it samples τ from the forward distribution (in our case, the inference model $q_{\phi, \psi}$). We discussed when this might be favorable in our setting in Appendix B (Equation 16). However, the joint matching loss as defined in Equation 8 is *off-policy*, as we sample from p and not from $q_{\phi, \psi}$.

E.2 Relation to common divergences

These questions open quite some design space, as was recently noted when comparing the trajectory balance loss to divergences commonly used in variational inference [34]. Redefining $P_F = q$ and $P_B = p$, the authors compare the trajectory balance loss with the KL-divergence and the reverse KL-divergence and prove that

$$\nabla_\phi D_{KL}(q || p) = \frac{1}{2} \mathbb{E}_{\tau \sim q} [\nabla_\phi \mathcal{L}(\tau)]. \quad (30)$$

That is, the *on-policy* objective minimizes the *reverse* KL-divergence between p and q . We do not quite find such a result for the *off-policy* version we use for the joint matching loss in Equation 8:

$$\nabla_\phi D_{KL}(p || q) = -\mathbb{E}_{\tau \sim p} [\nabla_\phi \log q] \quad (31)$$

$$\mathbb{E}_{\tau \sim p} [\nabla_\phi \mathcal{L}(\tau)] = -2\mathbb{E}_{\tau \sim p} [(\log p - \log q) \nabla_\phi \log q] \quad (32)$$

So why do we choose to minimize the joint matching loss rather than the (forward) KL divergence directly? This is because, as is clear from the above equations, it takes into account how far the ‘predicted’ log-probability $\log q$ currently is from $\log p$. That is, given a sample τ , if $\log p < \log q$, the joint matching loss will actually *decrease* $\log q$. Instead, the forward KL will increase the probability for every sample it sees, and whether this particular sample will be too likely under q can only be derived through sampling many trajectories.

Furthermore, we note that the joint matching loss is a ‘pseudo’ f-divergence with $f(t) = t \log^2 t$ [34]. It is not a true f-divergence since $t \log^2 t$ is not convex. A related well-known f-divergence is the Hellinger distance given by

$$H^2(p || q) = \frac{1}{2} \mathbb{E}_{\tau \sim p} [(\sqrt{p} - \sqrt{q})^2]. \quad (33)$$